Aula 08 – MIPS Pipeline: Solucionando Hazards de Dados

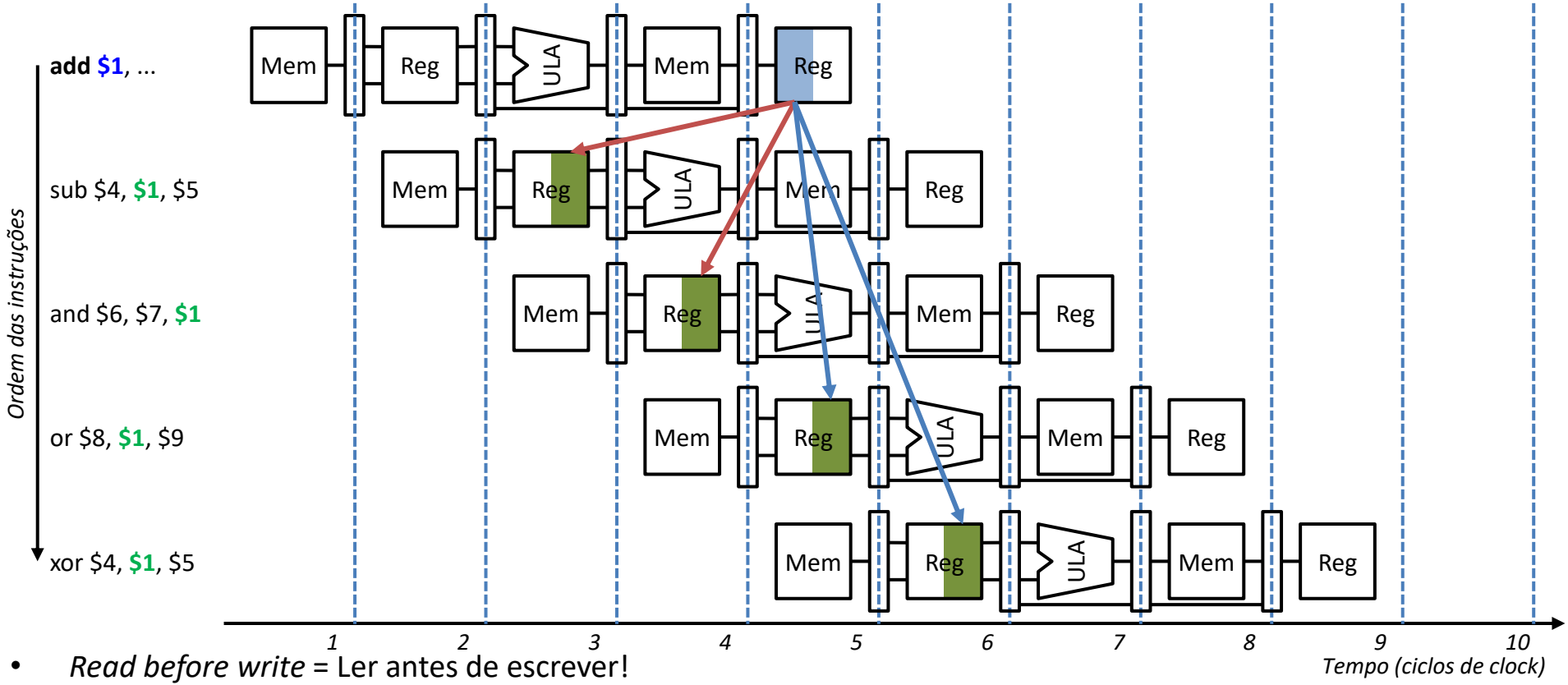
Prof. João Fernando Mari
joaofmari.github.io

Roteiro

- Hazards de Dados
  - Read-Before-Write
  - Load-Use
  - Solução simples (stall)
  - Solução por encaminhamento
- Solução por encaminhamento
  - Load-Use Hazards
  - Caminho de dados com detecção de hazards

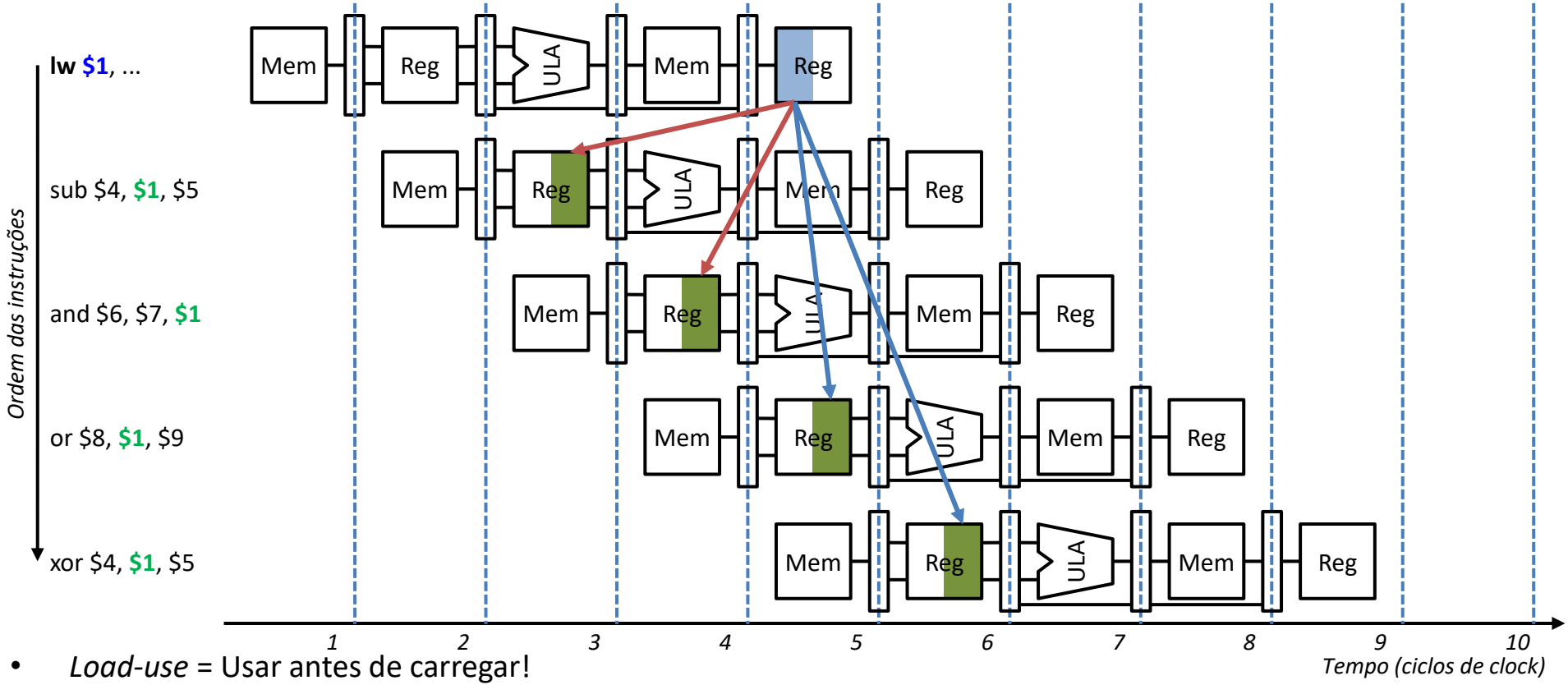
HAZARDS DE DADOS

Hazard de dados – *Read before write*



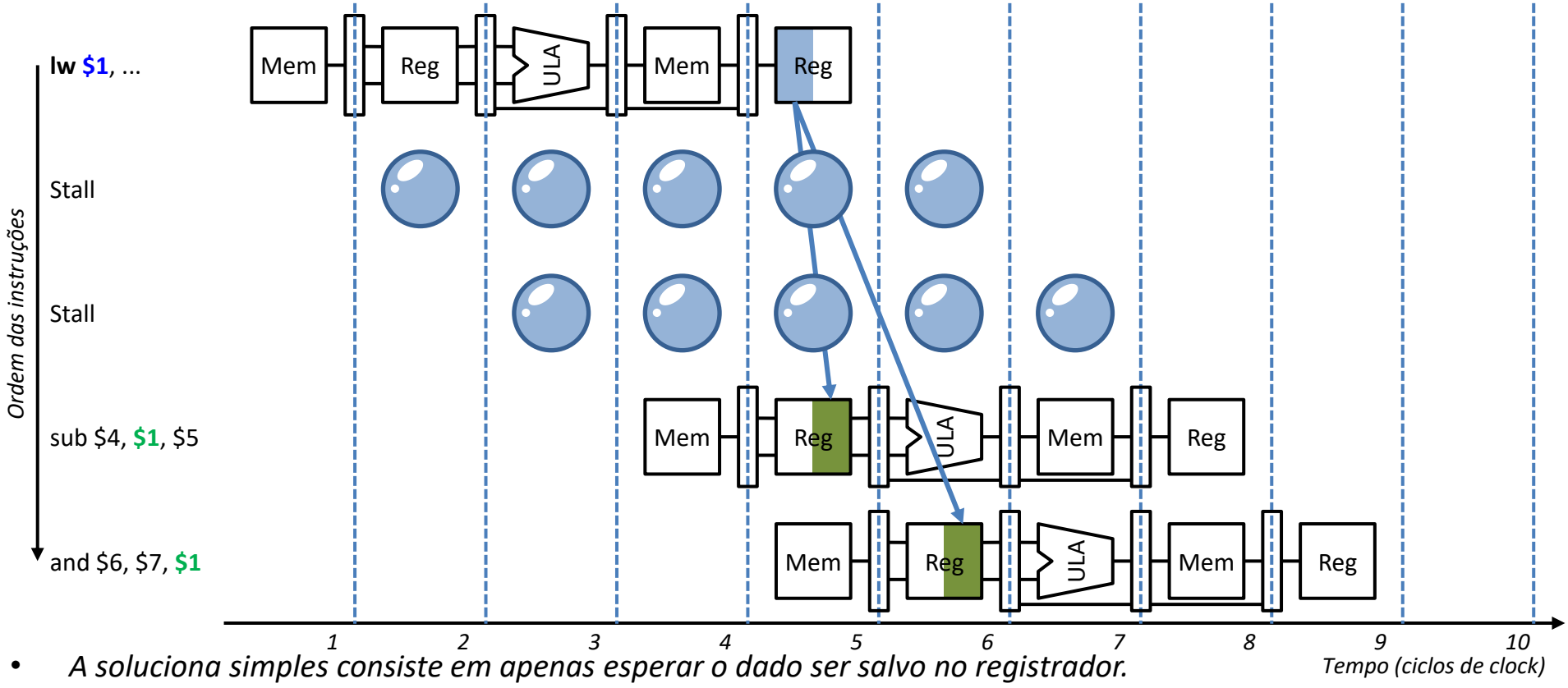
- *Read before write* = Ler antes de escrever!
  - Pode se gerado (iniciado) por instruções lógicas/aritméticas!

Hazard de dados – *Load-use hazard*



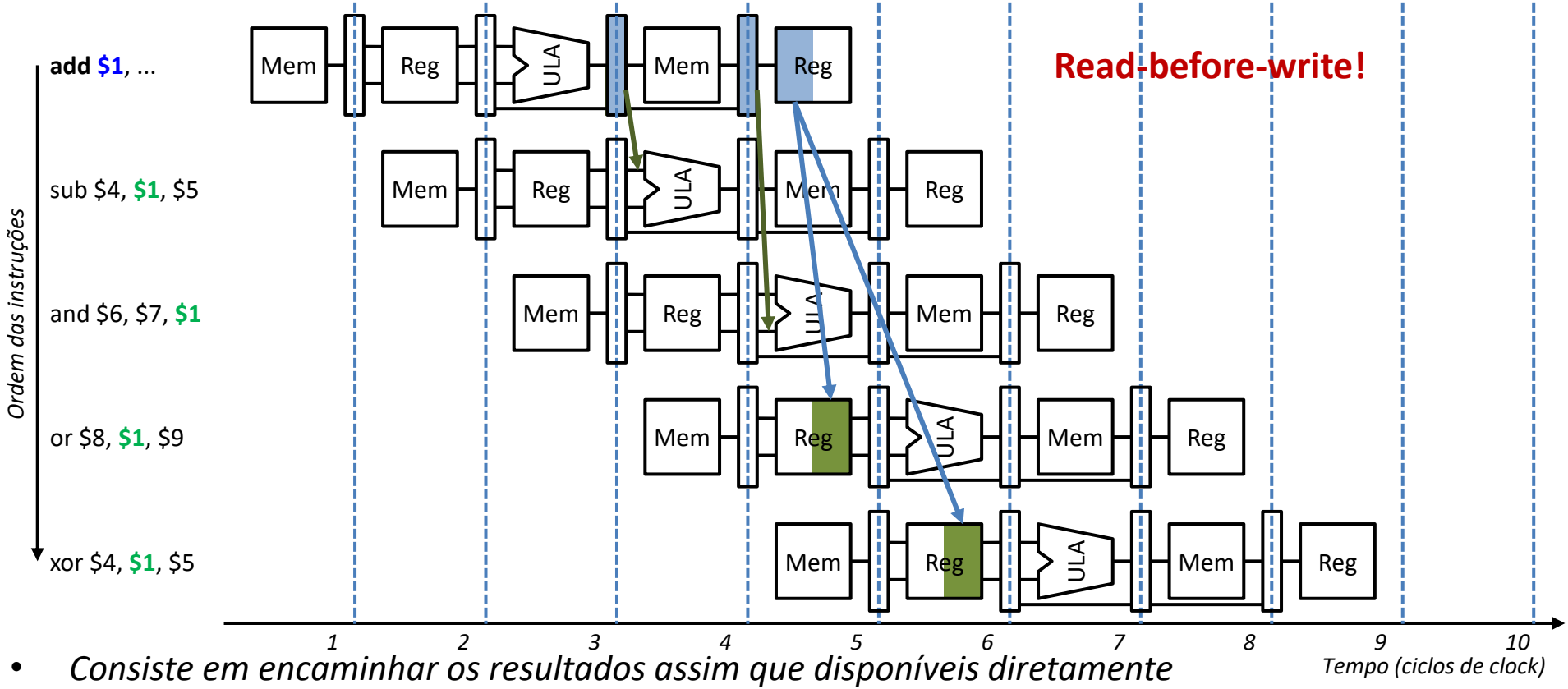
- *Load-use* = Usar antes de carregar!
  - Pode se gerado (iniciado) por load word (lw)!

Hazard de dados – Solução simples



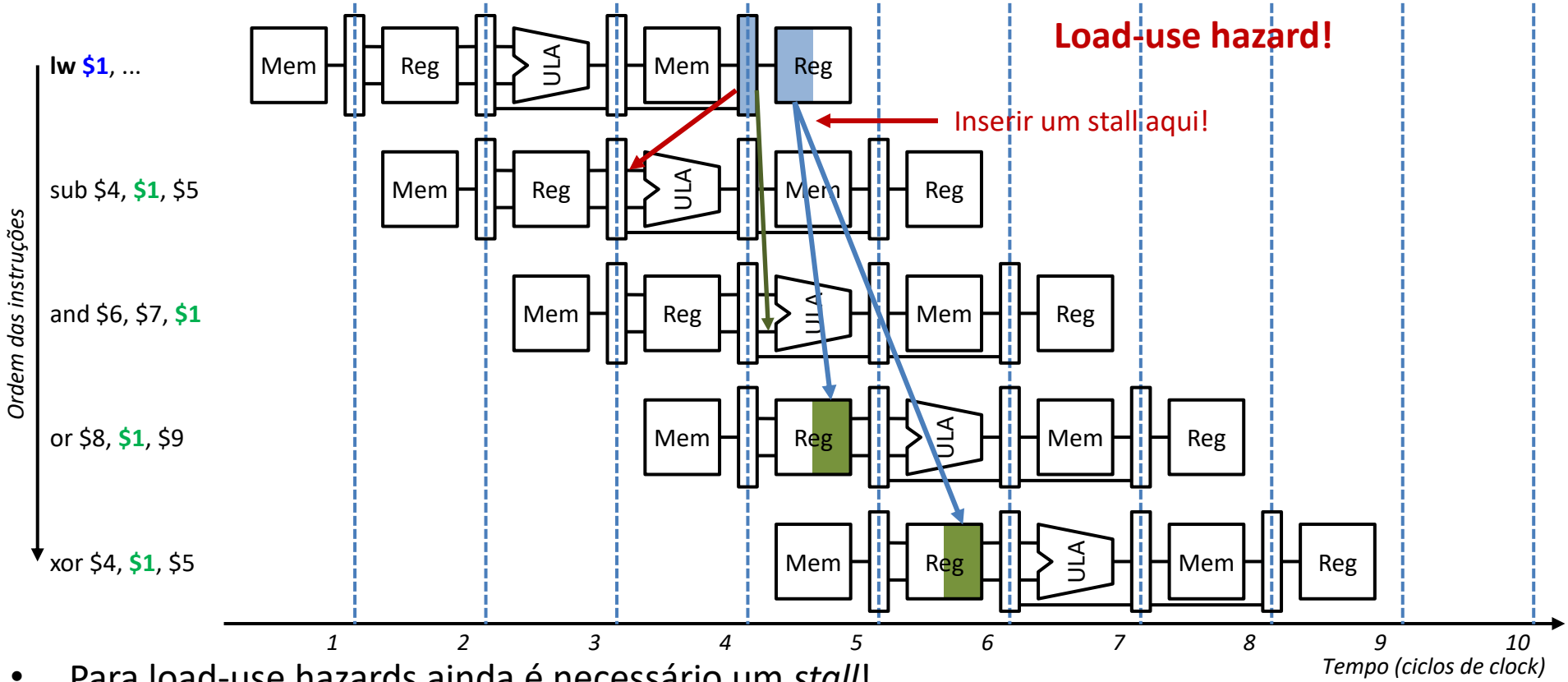
- *A soluciona simples consiste em apenas esperar o dado ser salvo no registrador.*
  - *Esta solução impacta o CPI, reduzindo o desempenho.*

Hazard de dados – Solução por encaminhamento



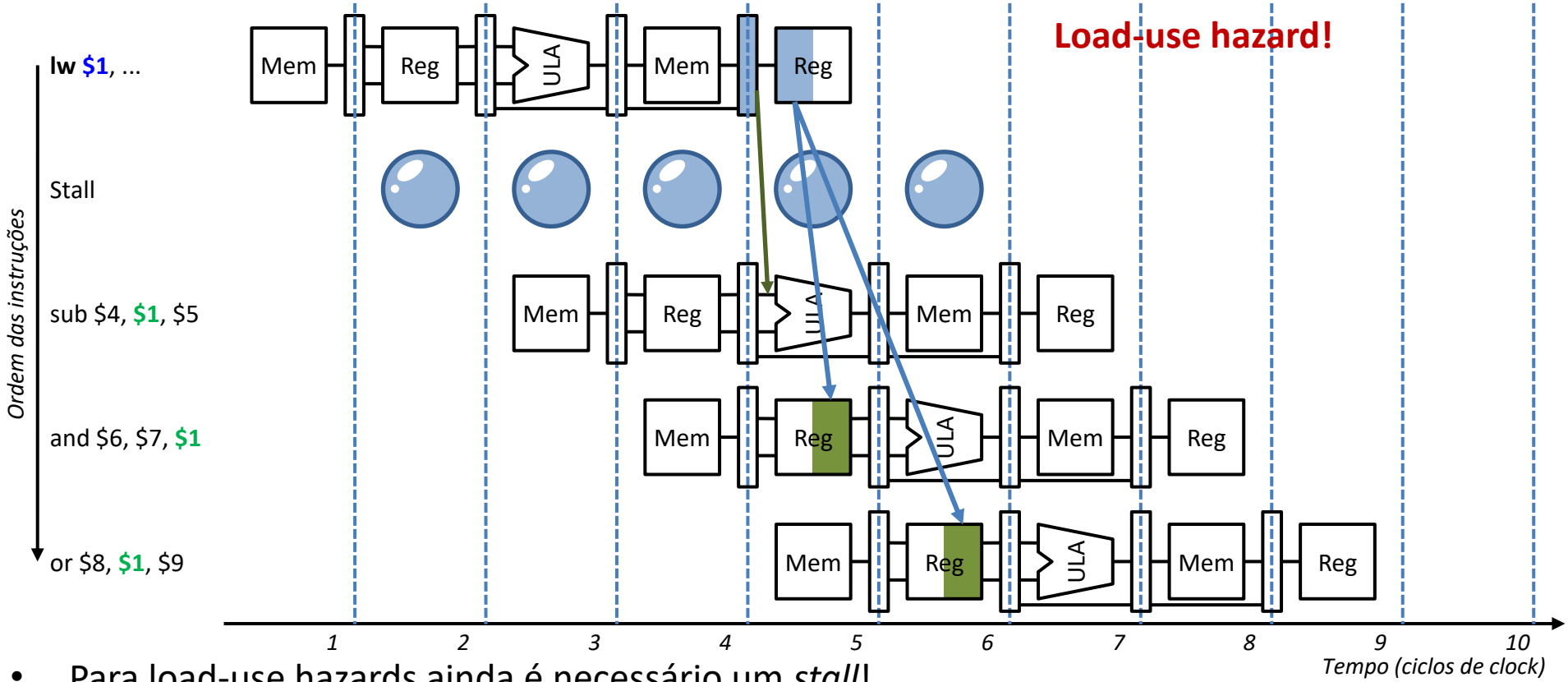
- *Consiste em encaminhar os resultados assim que disponíveis diretamente para onde forem necessários*

Hazard de dados – Solução por encaminhamento



- Para load-use hazards ainda é necessário um *stall*!

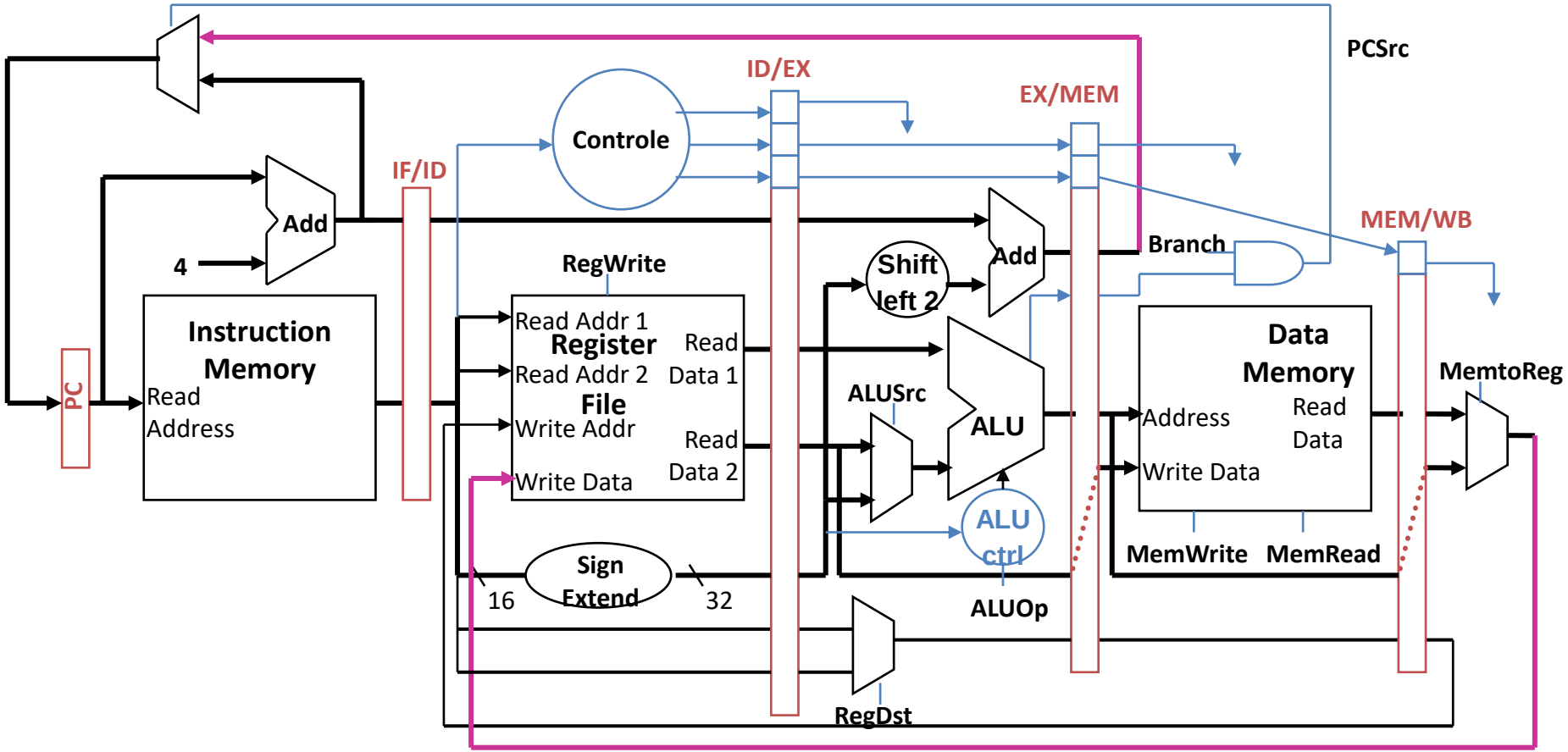
Hazard de dados – Solução por encaminhamento



- Para load-use hazards ainda é necessário um *stall*!

SOLUÇÃO POR ENCAMINHAMENTO

Solução por encaminhamento (forwarding)



Solução por encaminhamento (*forwarding*)

- A implementação do encaminhamento necessita de uma forma de identificar o hazard.
- Deve-se coletar o resultado dos registradores de pipeline e realizar o encaminhamento para a ALU que necessita dos dados:
  - Para a ALU, as entradas vem de quaisquer registradores de pipeline ao invés do registrador ID/EX
  - Multiplexadores são adicionados nas entradas da ALU para selecionados os dados corretos
  - Adiciona-se um controle de hardware apropriado para controlar os novos multiplexadores
  - Outras unidades funcionais, como DM, necessitam de uma lógica similar
- Com *forwarding*, é possível atingir um CPI igual a 1 mesmo quando existirem dependências de dados

Solução por encaminhamento (*forwarding*)

Ordem das instruções

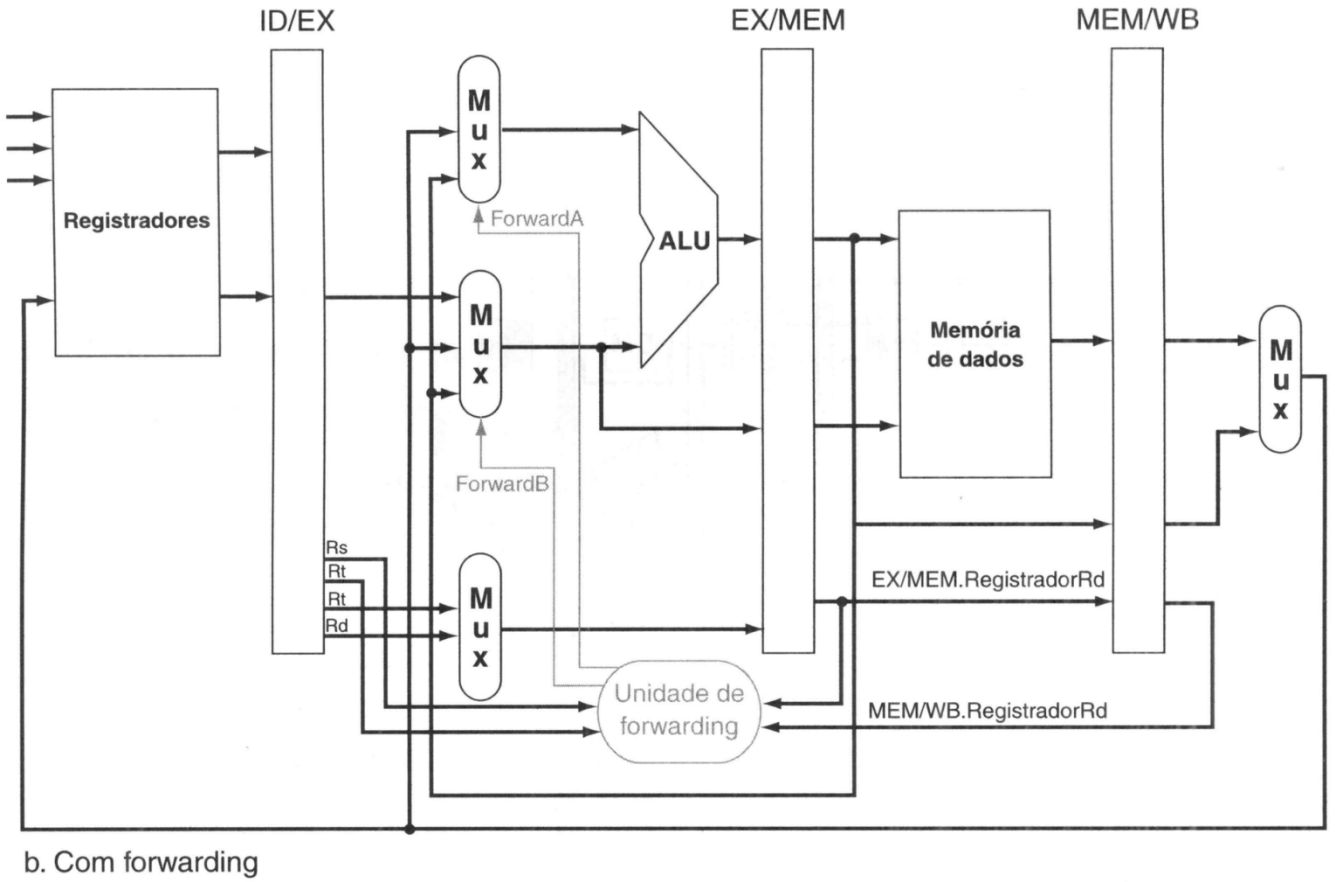
lw $1, ...

sub $4, **$1**, $5

and $6, **$1**, $7

or $8, **$1**, $9

xor $4, **$1**, $5



b. Com forwarding

Solução por encaminhamento (*forwarding*)

Controle do MUX	Origem	Explicação
ForwardA = 00	ID/EX	O primeiro operando da ULA vem do banco de registradores.
ForwardA = 10	EX/MEM	O primeiro operando da ULA sofre forwarding do resultado anterior da ULA.
ForwardA = 01	MEM/WB	O primeiro operando da ULA sofre forwarding da memória de dados ou de um resultado anterior da ULA.
ForwardB = 00	ID/EX	O segundo operando da ULA vem do banco de registradores.
ForwardB = 10	EX/MEM	O segundo operando da ULA sofre forwarding do resultado anterior da ULA.
ForwardB = 01	MEM/WB	O segundo operando da ULA sofre forwaring da memória de dados ou de um resultado anterior da ULA.

Solução por encaminhamento (*forwarding*)

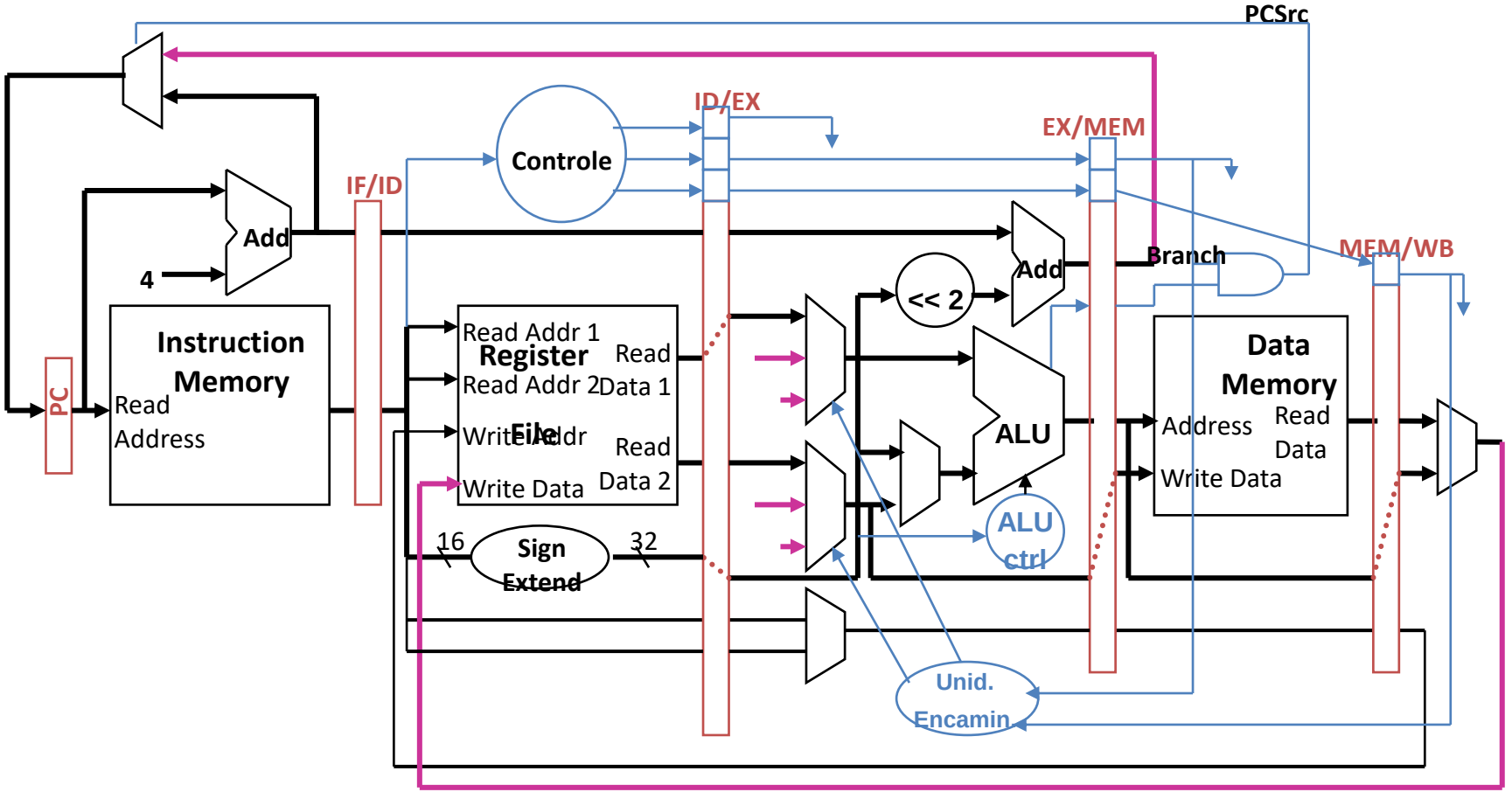
- **EX/MEM hazard:**

```
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
            ForwardA = 10
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
            ForwardB = 10
```

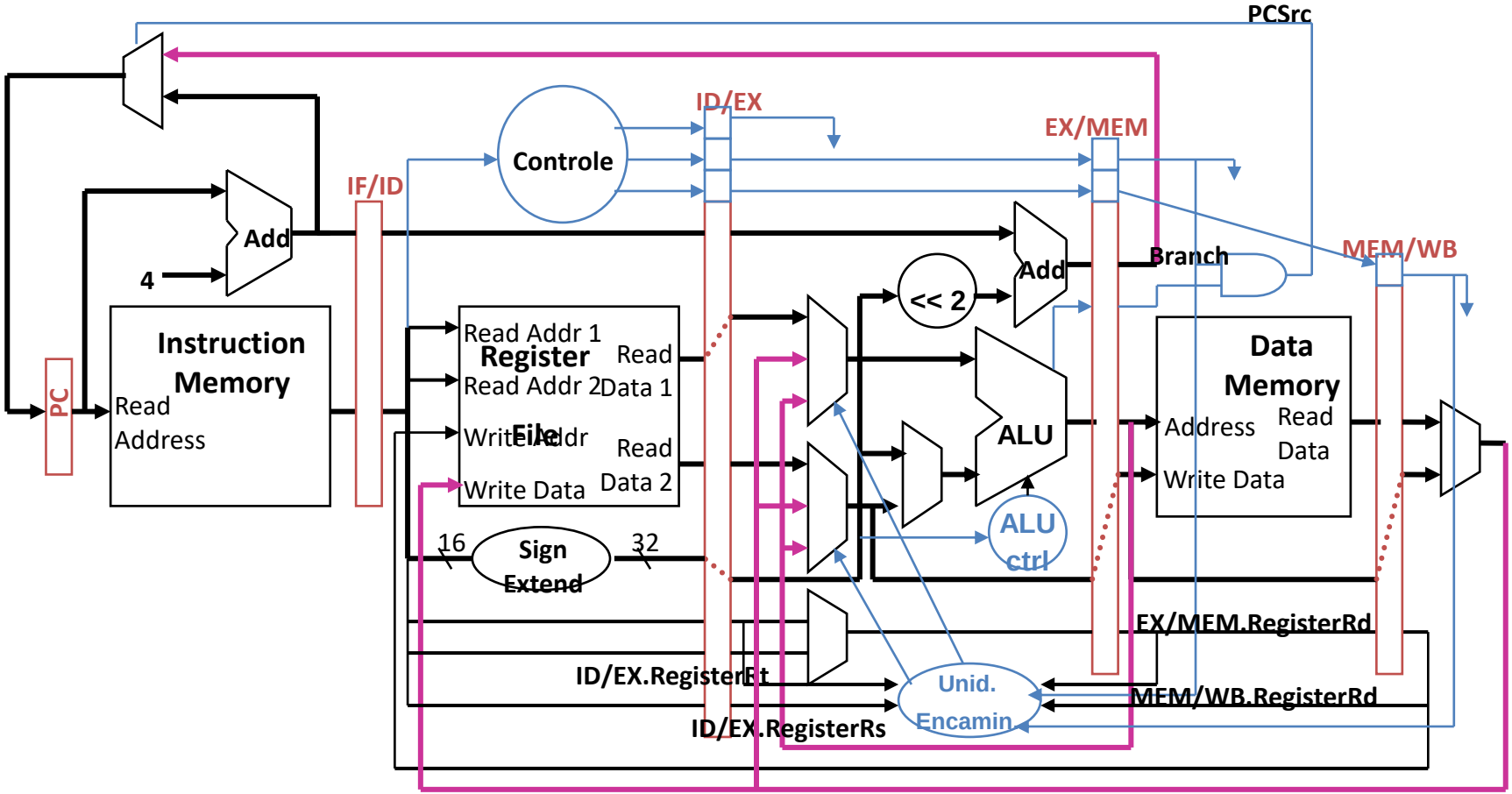
- **MEM/WB hazard:**

```
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
            ForwardA = 01
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
            ForwardB = 01
```

Solução por encaminhamento (*forwarding*)

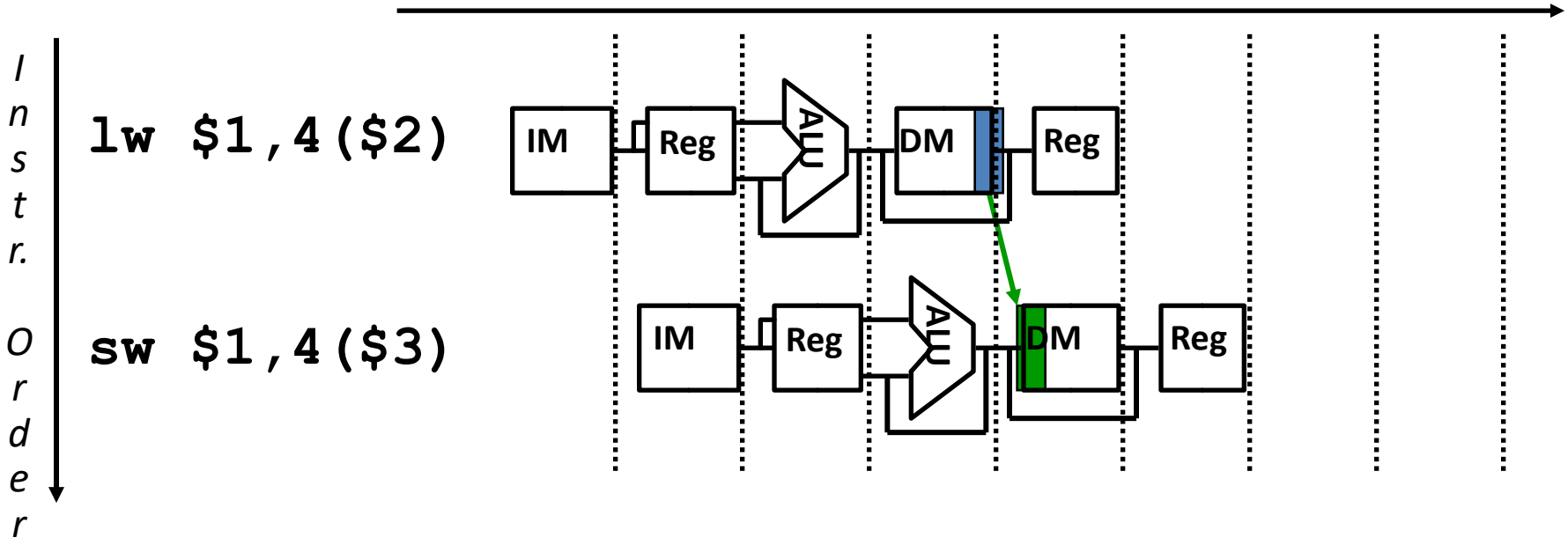


Solução por encaminhamento (*forwarding*)



Acessos a memória

- Loads imediatamente seguidos de stores
  - Stalls podem ser evitados adicionando hardware de encaminhamento a partir do registrador MEM/WB para a entrada da memória de dados.



LOAD-USE HAZARD

Load-Use Hazard

- Hardware de detecção de hazard no estágio ID:
  - Insere um stall entre a instrução load e o seu uso.
- Detecção de Hazard ID:

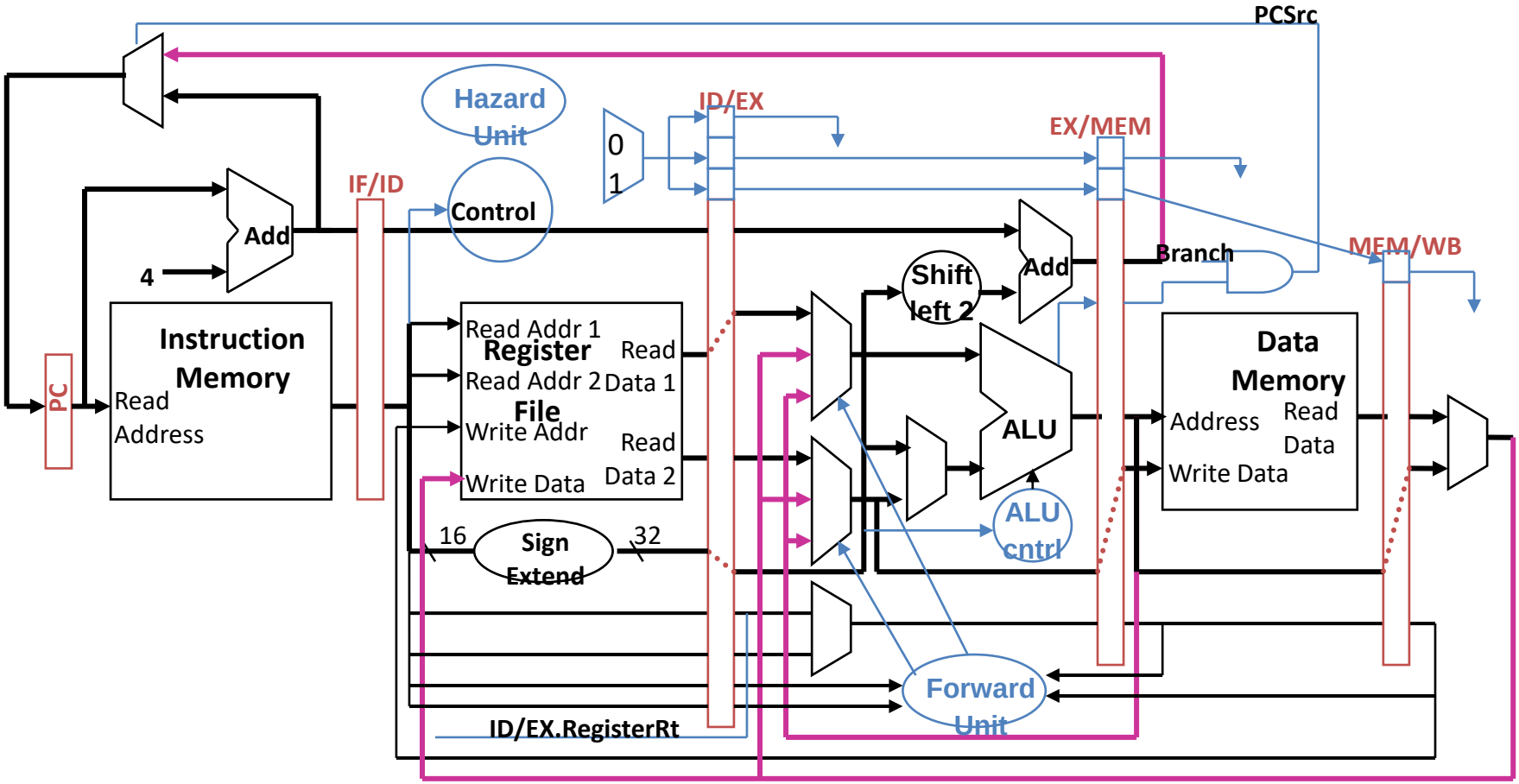
```
if (ID/EX.MemRead
    and ((ID/EX.RegisterRt = IF/ID.RegisterRs)
    or (ID/EX.RegisterRt = IF/ID.RegisterRt)))
        stall the pipeline
```

- A primeira linha verifica se a instrução no estágio de execução é um lw.
- As duas linhas seguintes verificam se o registrador de destino do lw casa com qualquer registrador fonte da instrução que está no estágio ID (instrução load-use).
- Após uma bolha de um ciclo, a lógica de encaminhamento pode corrigir os hazards de dados restantes.

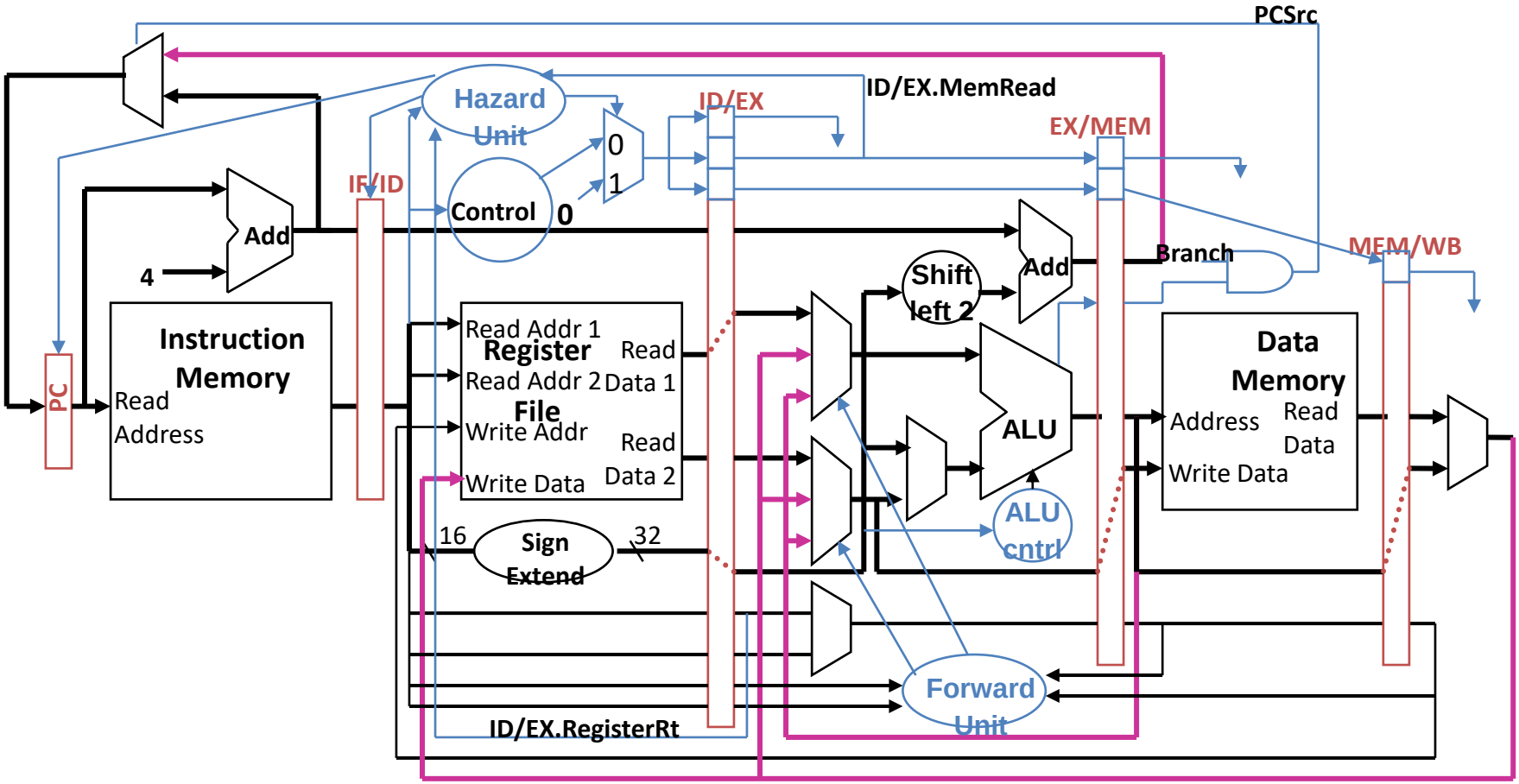
Load-Use Hazard

- A unidade de detecção de hazard controla a escrita para o PC e para os registradores IF/ID (PC.write e IF/ID.write)
  - Objetivo: Impedir que as instruções nos estágios IF e ID caminhem pelo pipeline.
- Insere uma bolha entre a instrução lw e a instrução no estágio ID (isto é, insere uma instrução nop)
  - Os bits de controle nos registradores EX, MEM, WB são setados como 0
  - A instrução após a instrução lw continua sua execução no pipeline

Caminho de dados com detecção de hazards



Caminho de dados com detecção de hazards



Bibliografia

- PATTERSON, D. A.; HENNESSY, J. L. **Organização e Arquitetura de computadores: a interface hardware/software**. 3. ed. Rio de Janeiro: Elsevier, 2005.
  - CAPÍTULO 5
- Notas de aula da Profa. Mary Jane Irwin
  - CSE 331 Computer Organization and Design
  - http://www.cse.psu.edu/research/mdl/mji/mjicourses

FIM